SPCIM: Sparsity-Balanced Practical CIM Accelerator With Optimized Spatial-Temporal Multi-Macro Utilization

Yiqi Wang, Fengbin Tu, *Member, IEEE*, Leibo Liu, *Senior Member, IEEE*, Shaojun Wei, *Fellow, IEEE*, Yuan Xie, *Fellow, IEEE*, and Shouyi Yin, *Member, IEEE*

***Abstract*— Compute-in-memory (CIM) is a promising technique that reduces data movement in neural network (NN) acceleration. To achieve higher efficiency, some recent CIM accelerators exploit NN sparsity based on CIM's small-grained operation unit (OU) feature. However, new problems arise in a practical multi-macro accelerator: The mismatch between workload parallelism and CIM macro organization causes spatial under-utilization; The multiple macros' different computation time leads to temporal under-utilization. To solve the under-utilization problems, we propose a Sparsity-balanced Practical CIM accelerator (SPCIM), including optimized dataflow and hardware architecture design. For the CIM dataflow design, we first propose a reconfigurable cluster topology for CIM macro organization. Then we regularize weight sparsity in the OU-height pattern and reorder the weight matrix based on the sparsity ratio. The cluster topology can be reshaped to match workload parallelism for higher spatial utilization. Each CIM cluster's workload is dynamically rebalanced for higher temporal utilization. Our hardware architecture supports the proposed dataflow with a spatial input dispatcher and a temporal workload allocator. Experimental results show that, compared with the baseline sparse CIM accelerator that suffers from spatial and temporal under-utilization, SPCIM achieves 2.94× speedup and 2.86× energy saving. The proposed sparsity-balanced dataflow and architecture are generic and scalable, which can be applied to other CIM accelerators. We strengthen two state-of-the-art CIM accelerators with the SPCIM techniques, improving their energy efficiency by 1.92× and 5.59×, respectively.**

***Index Terms*— Compute-in-memory (CIM), neural network, sparsity, CIM dataflow, CIM accelerator.**

I. Introduction

THE ubiquitous intelligent applications have motivated many neural network (NN) accelerators. However, the increasing NN modelsize makes conventional NN accelerators suffer from the massive data movement between multiply-accumulate (MAC) units and memory. Compute-in-memory (CIM) tackles this memory bottleneck by integrating MAC logic into SRAM, ReRAM, or other memory. Many CIM accelerators [16], [17], [19], [23], [24] have been designed in the past few years, proving CIM is a promising technique for efficient NN acceleration.

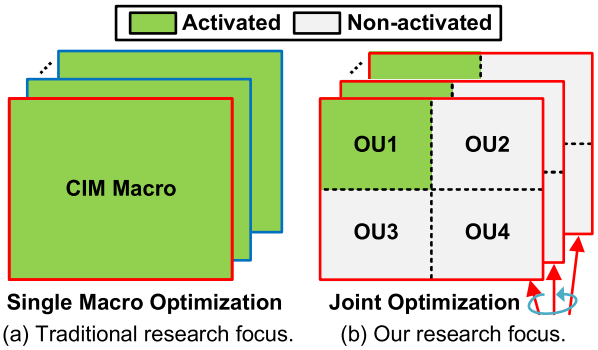


Fig. 1. CIM accelerator research focuses: (a) Traditional research usually focuses on optimizations for just a single CIM macro with an assumption of full macro activation. (b) Our research focuses on joint optimizations for multiple CIM macros, and considers partial macro activation (OU limitation).

Traditional CIM accelerators usually have an impractical assumption that the whole CIM macro can be activated in a single cycle, or focus on optimizations for just a single macro [2], [3], [15] (see Fig. 1(a)). However, a practical CIM accelerator has two important features: OU limitation and multi-macro architecture. First, due to the analog deviation and high-resolution ADC overhead, a practical CIM macro usually only activates limited numbers of rows and columns per cycle, leading to a smaller execution granularity called an operation unit (OU). Second, advanced CIM accelerators tend to integrate multiple CIM macros to avoid frequent weight data transfer and alleviate the memory wall issue [4], [5], [10]. Simply considering processing on a single macro is not enough to optimize the multi-macro acceleration. The increased CIM macro dimensions may cause mismatch with workload. As shown in Fig. 1(b), this paper focuses on a practical multi-macro CIM accelerator with OU limitation. Different macros are considered jointly for workload mapping, and each macro activates an OU per cycle.

Some recent CIM accelerators [26], [27], [28] take the OU limitation into account. To further improve the NN computing

Manuscript received 2 June 2022; revised 16 September 2022; accepted 15 October 2022. Date of publication 31 October 2022; date of current version 25 January 2023. This work was supported in part by the National Key Research and Development Program under Grant 2018YFB2202600, in part by the NSFC under Grant U19B2041 and Grant 61774094, in part by the Beijing Science and Technology (S&T) Project under Grant Z191100007519016, and in part by the Beijing Innovation Center for Future Chip. This article was recommended by Associate Editor W. Liu. *(Yiqi Wang and Fengbin Tu contributed equally to this work.) (Corresponding author: Shouyi Yin.)*

Yiqi Wang, Leibo Liu, Shaojun Wei, and Shouyi Yin are with the Beijing Innovation Center for Future Chip, Beijing National Research Center for Information Science and Technology, School of Integrated Circuits, Tsinghua University, Beijing 100084, China (e-mail: yinsy@tsinghua.edu.cn).

Fengbin Tu and Yuan Xie are with the Department of Electrical and Computer Engineering, University of California at Santa Barbara, Santa Barbara, CA 93106 USA.

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TCSI.2022.3216735.

Digital Object Identifier 10.1109/TCSI.2022.3216735

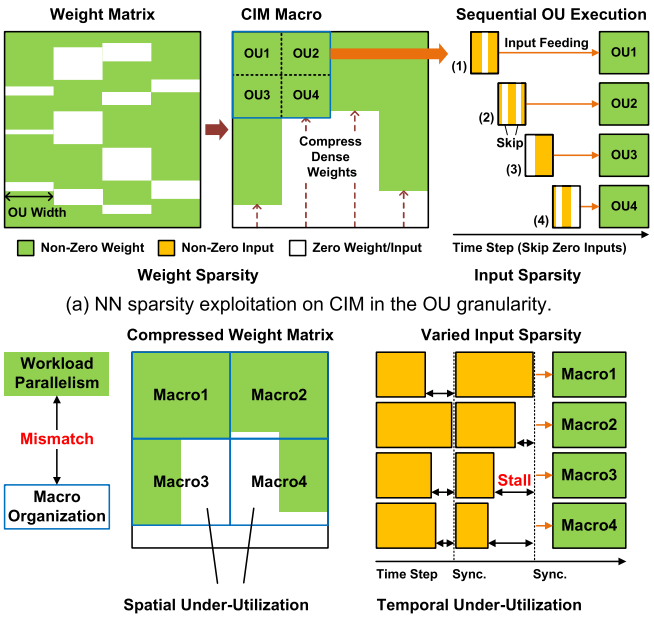


Fig. 2. NN sparsity exploitation on CIM in the OU granularity (an example with four OUs per macro), and under-utilization problems for a practical multi-macro CIM accelerator. The mismatch between workload parallelism and CIM macro organization causes spatial under-utilization. The macros' different computation time leads to temporal under-utilization.

efficiency, they exploit NN sparsity based on the partial activation feature of CIM macros. Fig. 2(a) illustrates how the OU granularity can be utilized to exploit NN sparsity. For weight sparsity, the sparsity pattern can be regularized according to the OU width or height. Only dense weights are compressed and stored in CIM macros to avoid unnecessary computation for sparse weights. For input sparsity, if the input bits sent to an OU are all zeros, the computation in this cycle is redundant and can be skipped for speedup. Although these works show NN sparsity can be exploited well on a single CIM macro, they do not consider multi-macro imbalance in a practical CIM accelerator, which limits resource utilization.

When OU limitation and multiple-macro architecture are both considered in a sparse CIM accelerator, new problems of CIM under-utilization arise, as shown in Fig. 2(b):

(1) Spatial under-utilization comes from the mismatch between workload parallelism and CIM macro organization. Different from digital accelerators whose weights can be distributed flexibly, weights in CIM accelerators are stored stationarily in CIM macros. Conventionally, CIM macros are organized in a fixed 2D-mesh topology to utilize the workload parallelism in the weight dimension. However, the parallelism gets reduced after the sparse weight matrix is compressed. If the weight matrix cannot be perfectly mapped onto CIM macros, spatial under-utilization occurs and leads to a waste of computing resources. Moreover, due to the random distribution of weight sparsity, the matrix shape becomes irregular after compression, making the mismatch problem more severe.

(2) **Temporal under-utilization** results from the macros' different computation time. Since CIM macros are fed with different inputs, the varied input sparsity leads to diverse computation time for the parallel macros. Thus, the overall computation time is limited by the macro with the densest inputs. The other macros have to stall for synchronization, and then update weights together for a new round of computation. The macro stall results in temporal under-utilization.

To solve the above CIM under-utilization problems, we propose a Sparsity-balanced Practical CIM accelerator (SPCIM), including dataflow design and hardware architecture. For the CIM dataflow design, we first propose a reconfigurable cluster topology for CIM macro organization. Then we choose the OU-height pattern for weight sparsity regularization and reorder the weight matrix based on the sparsity ratio. Based on those designs, we propose spatial and temporal workload mapping as the optimized SPCIM dataflow. The SPCIM architecture provides hardware support for the dataflow optimization techniques. The followings are the detailed technical contributions of this work:

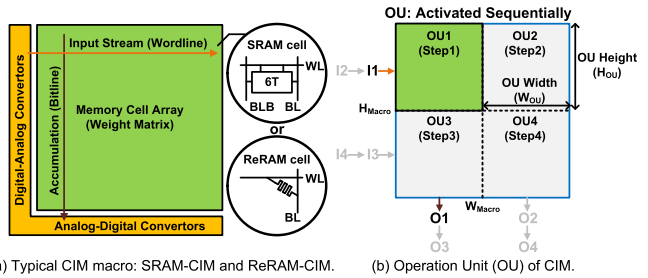
- For spatial under-utilization, the SPCIM dataflow provides optimized spatial workload mapping: The compressed weight matrix is reordered and divided into sub-matrices. The CIM cluster topology is reconfigured for each sub-matrix to match workload parallelism, with the best clustering factor to maximize spatial utilization.
- For **temporal under-utilization**, the SPCIM dataflow provides optimized temporal workload mapping: Different from the conventional 2D-mesh topology, the macros in one cluster are organized in a row, and share the same inputs with the help of OU-height weight sparsity. As a result, we guarantee the macros in one cluster always have the same computation time. Moreover, we dynamically assign inputs to different CIM clusters according to their computation status, thus reducing computation stall before synchronization.
- We design the SPCIM architecture to support the above sparsity-balanced CIM dataflow, featuring a spatial input dispatcher and temporal workload allocator. The spatial input dispatcher collects inputs for the reordered weights and enables CIM cluster topology reconfiguration. The temporal workload allocator monitors the cluster's computation status and realizes dynamic input allocation.

The rest of this paper is organized as follows: Section II discusses related work. Section III proposes the SPCIM dataflow. Section IV describes the SPCIM architecture details. Section V presents experimental results. Section VI concludes this paper.

II. Related Work

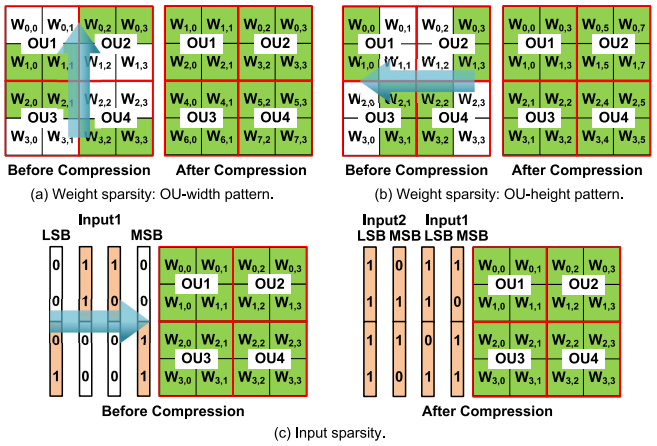
A. Practical CIM Feature1: OU Limitation

The first feature of practical CIM accelerators is OU limitation. By implementing MAC logic in memory, CIM eliminates the boundary between computation and memory, which is an attractive solution for efficient NN acceleration. CIM is usually designed on the basis of SRAM [16], [17], [19] or ReRAM [1], [9], [22], [23], [24], as illustrated in Fig. 3(a). The 2D cell array stores a weight matrix. Inputs are converted to analog signals and streamed through the wordlines. In the CIM macro, inputs and weights perform analog bit-wise MAC operations through current or voltage. Partial sums are accumulated on



(a) Typical CIM macro: SRAM-CIM and ReRAM-CIM. (b) Operation Unit (OU) of CIM.

Fig. 3. CIM architecture and OU limitation. (a) A Typical CIM macro. Weights are stored in a 2D memory cell array and perform bit-wise MAC operations with streamed inputs. (b) Operation Unit (OU) of CIM. OU size is the macro activation granularity per cycle.



(a) Weight sparsity: OU-width pattern. (b) Weight sparsity: OU-height pattern. (c) Input sparsity.

Fig. 4. Examples of OU-level sparsity. (a) Weight sparsity: OU-width pattern, regularized according to the OU width and compressed along the vertical direction. (b) Weight sparsity: OU-height pattern, regularized according to the OU height and compressed along the horizontal direction. (c) Input sparsity, which skips all-zero inputs to an OU and compresses dense inputs.

the bitlines. The outputs are converted back to digital signals in the end.

Since most CIMs are based on analog computing, only a limited number of wordlines and bitlines can be activated concurrently in a practical CIM macro [1], [12], [22], [26]. The maximum number of wordlines that can be activated concurrently depends on the accuracy constraint. Turning on too many wordlines will cause large accumulated analog deviation on the bitline, leading to overlap with neighboring states. Therefore, it is difficult for the ADCs to read out the accurate partial sum accumulation results. The read out error will dramatically degrade the inference accuracy. For instance, even making a positive estimation for ReRAM technology advance that enables the R-ratio/resistance-deviation to increase/shrink by 3× of the WOx ReRAM [7], the inference accuracy of modern NNs will drop considerably when more than 16 wordlines are activated simultaneously. Besides the constraint on the number of concurrently activated wordline, the activated bitline count is limited by the number of ADCs. To save the large area and power overhead of ADCs, usually multiple columns share one ADC on the output side of CIM. Thus, only a limited number of bitlines can be operated in a single cycle, and bitlines reuse the ADCs in different time.

As a result, practical CIM usually works in a smaller granularity than the entire macro, called an operation unit (OU). For example, in a state-of-the-art 1024 × 512 ReRAM-CIM macro [24], the OU granularity is only 4×64. As demonstrated in Fig. 3(b), the OUs of a CIM macro are activated sequentially, generating outputs O1∼O4 in different time steps.

B. Practical CIM Feature2: Multi-Macro Architecture

The second feature of practical CIM accelerators is multi-macro architecture. Building a multi-macro CIM system is a practical approach to improving overall system performance and energy efficiency. Integrating more macros increases the accelerator's CIM capacity and also boosts the overall throughput. Moreover, by storing more weights on chip, there's less chance to halt computation and wait for weight reloading into CIM macros [19].

Many CIM accelerators are based on multi-macro architectures, but they don't optimize the multi-macro organization. For instance, ISAAC [15] and SRE [26] have 96 macros with 128 × 128 array size in each PE to build a large-scale ReRAM-CIM accelerator. ISSCC'21-15.2 [27] incorporates 4 macros to realize a system-level sparsity-aware CIM accelerator. However, their CIM macros are organized in a fixed topology, which cannot fit the diverse workload parallelism in different benchmarks and different layers. Some CIM macros cannot be fully mapped with weights in small workload parallelism, which occurs frequently especially in modern NNs like DenseNet [8]. There's a trend of modern NNs that employ smaller weight matrix sizes and add bottleneck layers, so as to reduce the number of parameters. In these cases, the fixed macro topology results in severe spatial under-utilization.

C. NN Sparsity Exploitation on Practical CIM

Some recent CIM accelerators [26], [27], [28] consider the OU limitation for sparsity exploitation. Although the OU limitation leads to partial activation of a CIM macro, the smaller execution granularity also offers an opportunity for OU-level sparsity acceleration. They regularize weight sparsity according to the OU width or height, and detect OU's zero input bits to skip cycles. In Yue et al.'s SRAM-CIM accelerators, ISSCC'20-14.3 [28] skips all-zero inputs to save execution cycles, and ISSCC'21-15.2 [27] stores the weight matrix in a compressed way to skip zero-weight computation. SRE [26] is a ReRAM-CIM accelerator with sparsity exploitation methods similar to Ref. [27].

Fig. 4 shows several examples of the OU-level sparsity exploitation for weights and inputs, where each CIM macro has 2 × 2 OUs and each OU has × weights. Weight sparsity can be exploited in the OU-width pattern or OU-height pattern. In Fig. 4(a), all the weights of the 1^{st} row vector in OU1, the 2^{nd} row vector in OU2, the 2^{nd} row vector in OU3, and the 1^{st} row vector in OU4 are regularized to zeros. In this OU-width pattern, zero rows are removed within each OU and the remaining dense rows are shifted up. In the OU-height pattern (see Fig. 4(b)), zero columns are removed and the remaining columns are shifted left. Similarly, as shown in Fig. 4(c), when the input bits concurrently fed to one OU's

TABLE I
TERM DEFINITION FOR CIM ARCHITECTURE AND INPUT GRANULARITY

Term	Definition
Top-to-down. CIM Cluster > CIM Macro > CIM OU. Input Tile > Input Brick.	
CIM Cluster	α CIM macros organized in a row. α: Clustering Factor.
CIM Macro	A CIM cell array with H_{Macro} rows and W_{Macro} columns. Each row stores w weights. $w = W_{Macro}$/weight precision.
CIM OU	Macro activation granularity. A CIM macro activates H_{OU} rows and W_{OU} columns per cycle.
Input Tile	All inputs to the current sub-matrix are partitioned to different input tiles, which have H_{Macro} bits per cycle.
Input Brick	H_{OU} input bits that are concurrently fed to an OU in a cycle.

two wordlines are all zeros, they are skipped and the dense inputs are compressed. In this way, we can process more inputs within the same amount of time.

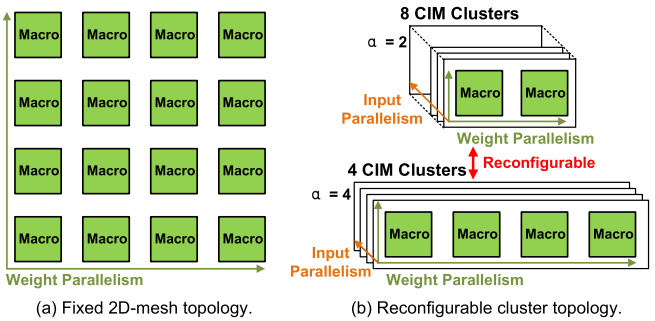
However, they only focus on the sparsity processing of a single macro. Besides the spatial under-utilization from the fixed macro organization, sparsity exploitation further raises new temporal imbalance problems across different macros. As pointed out in Fig. 2(b), **spatial and temporal under-utilization** appears in a practical multi-macro CIM accelerator. The mismatch between workload parallelism and CIM macro organization causes spatial under-utilization. The macros' different computation time leads to temporal under-utilization. In this paper, we will solve the problems in both the dataflow and architecture levels (Section III and IV).

III. SPARSITY-BALANCED SPCIM DATAFLOW

In order to solve the under-utilization problems of multi-macro CIM accelerators, we propose a sparsity-balanced CIM dataflow. The dataflow design is divided into three steps for discussion: CIM macro organization, sparsity pattern selection, and workload mapping. CIM macro organization enables a reconfigurable cluster topology to fit workload parallelism. Sparsity pattern selection chooses the OU-height pattern to ensure the same computation time in one cluster. Based on the optimized macro organization and sparsity pattern, spatial and temporal workload mapping elaborates the details of how to map workload with the optimized SPCIM dataflow to solve the spatial and temporal under-utilization problems. For better illustration, we list the term definition for different granularities of CIM architecture and inputs in TABLE I.

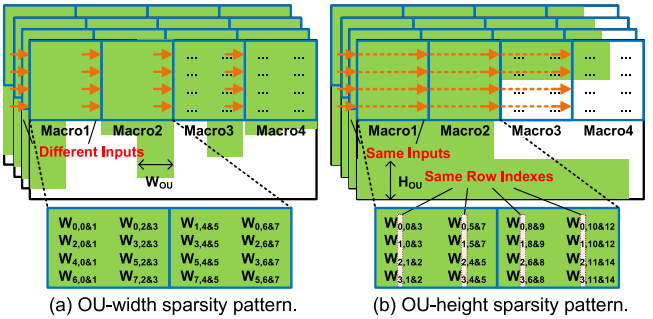
A. CIM Macro Organization

Conventionally, CIM macros are organized in a fixed 2D-mesh topology (see Fig. 5(a)). When this topology matches the workload parallelism in the vertical and horizontal weight dimensions well, there's no much spatial under-utilization for dense NN acceleration. However, the fixed topology cannot fit the diverse workload parallelism. Moreover, as shown in Fig. 2(b), weight sparsity leads to reduced and irregular weight parallelism, making it difficult to match the fixed 2D topology and fully utilize the CIM macros.



(a) Fixed 2D-mesh topology. (b) Reconfigurable cluster topology.

Fig. 5. CIM macro organization: (a) Fixed 2D-mesh topology. (b) Reconfigurable cluster topology. The α macros in one cluster are organized in a row, and different clusters store the same weights. α is the variable clustering factor. The reconfigurable topology is easier to match the available workload parallelism, due to the flexible exploitation of input and weight parallelism.



(a) OU-width sparsity pattern. (b) OU-height sparsity pattern.

Fig. 6. Sparsity pattern selection: (a) OU-width weight sparsity. Different macros require different inputs and their varied computation time leads to temporal under-utilization. (b) OU-height weight sparsity, which guarantees the same computation time in a cluster and avoids temporal under-utilization.

We propose a reconfigurable cluster topology for CIM macro organization. As illustrated in Fig. 5(b), the macros in one cluster are organized in a row, and different clusters store the same weights. We denote the number of macros per cluster as the clustering factor (α). The topology can be reconfigured according to the available workload parallelism, with a variable clustering factor from 1 to the total number of macros. The cluster reconfiguration offers an opportunity to make adaptive exploitation of input parallelism and weight parallelism, so we can find a good match for the compressed weight matrix by selecting an appropriate clustering factor. For example, when the weight matrix size is rather small, we can decrease the clustering factor to reduce the exploitation of weight parallelism and extend more input parallelism.

B. Sparsity Pattern Selection

Fig. 6 compares two kinds of OU-based weight sparsity patterns. In the OU-width pattern, weight sparsity is regularized according to the OU width, and compressed along the vertical direction. However, the random sparsity distribution makes the macros store different weight rows and require different corresponding inputs. The varied computation time of different inputs leads to temporal under-utilization.

The OU-height pattern offers an advantage in solving this problem. Since the weight sparsity is regularized according to

the OU height and compressed along the horizontal direction, weights do not move in the vertical direction. Weight row indexes of macros in one cluster remain unchanged after horizontal compression. By organizing the CIM macros in the previously discussed cluster topology, we can guarantee the macros in one cluster always have the same row indexes. Consequently, inputs to one weight row are identical. Macros in one cluster have the same inputs and computation time, so there's no temporal under-utilization within a cluster. The only problem left for temporal under-utilization is how to achieve temporal balancing across different clusters, which will be discussed in the following workload mapping methods.

Although the pruning method for NN model has been limited, our pruning method can deliver more speedups compared with the conventional element-wise pruning. We prune the model according to the OU-height sparsity in a fine granularity of $H_{OU} \times 1$, which has little impact on the pruning efficiency. With the help of iterative retraining and finetuning, ResNet-50 can also be pruned to 50% sparsity like element-wise pruning with no accuracy loss. When the sparsity ratio is high, the element-wise sparsity can prune to 85% weights with 2.04% accuracy loss and our pruning method can prune 80% weights under the same accuracy. However, the element-wise sparsity can be hardly utilized by CIM accelerators due to the rigid crossbar architecture, while CIM accelerators can gain more speedups benefiting from our pruning method. The baseline CIM accelerator can only achieve 1.09× and 1.15× speedups from the element-wise sparsity, under no accuracy loss and 2.04% accuracy loss respectively. If the OU-height pruning is directly applied, the speedups grow to 1.31× and 1.67×. With the proposed sparsity balance approach (elaborated in Section III-C), SPCIM finally achieves 1.78× and 3.65× speedups in the two cases.

C. Workload Mapping

In the previous two steps, we propose a reconfigurable CIM cluster topology together with the OU-height weight sparsity pattern. On this basis, we will explain how to map workload with our dataflow to solve the spatial and temporal under-utilization problems.

1) Spatial Workload Mapping: As shown in Fig. 7(a), the weight matrix is divided into several sub-matrices according to the CIM macro height. They are row-wisely traversed and mapped onto CIM clusters. However, due to the irregular sparsity distribution of pruned NN models, each weight group has different numbers of dense weights. After compressing dense weights and storing them in CIM, spatial imbalance occurs between different rows of OUs in one cluster. Some rows of OUs cannot be fully mapped with weights, leading to a waste of computing resources.

In Fig. 7(b), we reorder the weight matrix based on the number of dense weights in each weight group. Therefore, different rows of OUs in one cluster have similar weight parallelism. We reorder the weight matrix based on the sparsity ratio in the OU-height granularity, leading to similar weight parallelism for different rows of OUs in a cluster. In Fig. 7(b), weight groups (①;②;③;④) are reordered to weight groups

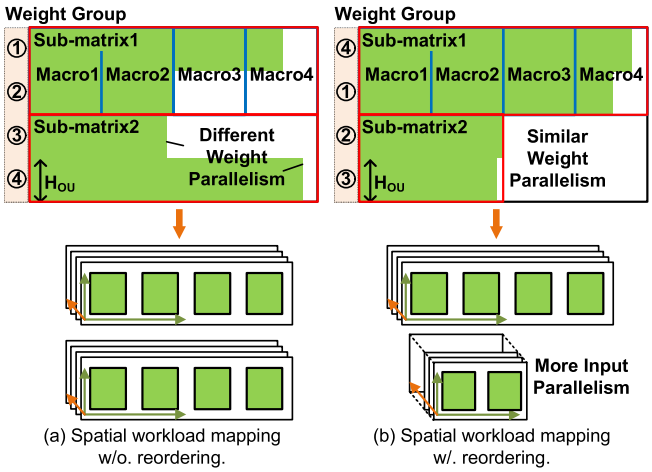


Fig. 7. Spatial workload mapping based on weight sparsity. After reordering weights in the OU-height granularity, our method can reconfigure the cluster topology for each sub-matrix to maximize spatial utilization.

(④;①;③;②). Each weight group contains H_{OU} weight rows. The cluster reconfiguration offers an opportunity to make adaptive exploitation of input parallelism and weight parallelism. Consequently, we can reshape the cluster topology for each sub-matrix to maximize spatial utilization. In the example, the topology is configured as $1 \times 4 \times 4$ for the first sub-matrix, and $1 \times 2 \times 8$ for the second sub-matrix. After weight reordering and cluster reconfiguration, the workload imbalance of rows of OUs in one cluster is alleviated, delivering higher spatial utilization. In addition, macros in one cluster still have the same computation time after reordering. Since one entire row of the weight matrix is reordered simultaneously, row indexes are still the same across different macros. By switching input positions correspondingly (from ①;②;③;④ to ④;①;②;③), the four macros still have the same inputs.

2) Temporal Workload Mapping: Fig. 8 shows an example of temporal workload mapping on four clusters. All inputs to the current weight sub-matrix are divided into multiple tiles that fit the cluster's local storage. Each input tile has the same number of inputs, but has different computation time due to the varied input sparsity. Temporal workload mapping describes how the input tiles are sequentially processed by the clusters. In the conventional method (static input allocation), each cluster is assigned the same number of input tiles. However, this is unaware of the varied input sparsity and different computation time, leading to computation stall and temporal under-utilization across clusters.

Thanks to the proposed CIM cluster topology, different clusters store the same weights and their input tiles can be directly exchanged. We take this opportunity to realize dynamic input allocation, which decides the destination of input tiles based on real-time cluster computation status. Once a cluster consumes the current input tile, we will assign a new input tile to this idle cluster. For example in Fig. 8(c), Cluster4 completes Tile8 while Cluster1∼3 are still processing their current workload. Then it continues to process Tile9. As a result, our method dynamically balances the cluster workload and improves temporal utilization.

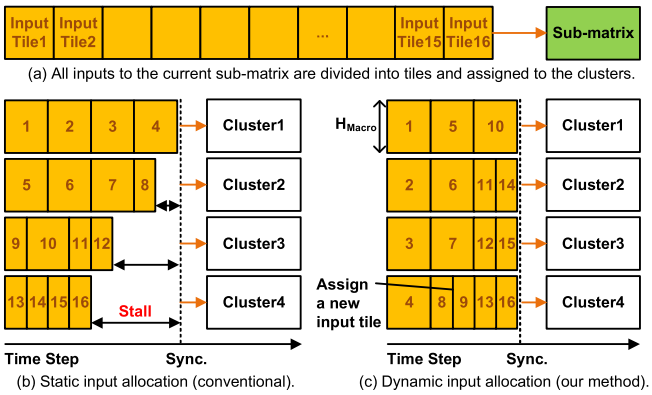


Fig. 8. Temporal workload mapping based on input sparsity. We dynamically assign input tiles based on real-time cluster computation status, reducing computation stall and improving temporal utilization.

IV. SPCIM Architecture Design

To support the proposed sparsity-balanced CIM dataflow, we design the SPCIM architecture as illustrated in Fig. 9(a), which consists of a top controller, a global input buffer, a spatial input dispatcher, a temporal workload allocator, 16 CIM cores, a SIMD core, and a global output buffer. Each CIM core contains a CIM macro that stores compressed and reordered weights. The spatial input dispatcher is composed of two parts: a selection unit and a distribution network. The selection unit collects input tiles required by the reordered weights. The distribution network realizes cluster topology reconfiguration by multicasting the same input tile to the cores in one cluster. The temporal workload allocator monitors the computation status of each cluster and dynamically assigns new input tiles to idle clusters. The results from CIM clusters are post-processed in the SIMD core, and finally stored in the global output buffer.

In this section, we first explain the OU-height input and weight sparsity exploitation in a CIM core. Then, we illustrate the spatial input dispatcher and temporal workload allocator, which support temporal and spatial workload mapping to solve the spatial and temporal under-utilization problems. Finally, we demonstrate how an NN model is mapped onto SPCIM, including layer mapping and work flow.

A. CIM Core

Fig. 9(b) presents the CIM core that exploits input sparsity and OU-height weight sparsity selected in Section III-B. Besides a CIM macro and local buffers, we design a real-time sparse encoder and sparsity-aware ping-pong accumulators for input and weight sparsity respectively.

1) Input Sparsity: Since input sparsity is dynamic and unknown before execution, we design a real-time sparse encoder to extract dense bricks from the input tile. A brick refers to the OU-height (H_{OU}) input bits that are concurrently fed to an OU per cycle. The sparse encoder fetches macro-height (H_{Macro}) bits from the local input buffer, and detects the sparsity of each brick using an OR gate. After detection, the dense brick together with the targeted OU row number and its magnitude is pushed into a FIFO. The targeted OU row number indicates the corresponding row of OUs for this brick and controls the activated wordlines. The magnitude represents the dense brick's bit position in the original input and controls the left-shift bits before accumulation. After a FIFO entry is popped, the macro serially activates the OUs in the targeted row, and sends the results to the sparsity-aware ping-pong accumulators.

2) Weight Sparsity: We design sparsity-aware ping-pong accumulators for sequential partial sum accumulation and output merging. In each cycle, the current OU's outputs are selected and left-shifted according to the input magnitude sent from the FIFO in the sparse encoder. Then, the shifted outputs are sent to one of the ping-pong accumulators (e.g., Accumulator1) for partial sum accumulation. Each accumulator has a decomposable adder and a partial-sum register. Since an OU has varied output bitwidth under different weight precision, the adder can be decomposed into small adders to support different accumulation bitwidth. During the processing of serially fed inputs, the shifted outputs are added to the partial sum and written to the register successively. When the current OU completes partial sum accumulation, the final results are stored in the partial sum register. The next step is to write the stored results to the local output buffer to combine partial sums, so Accumulator1 transforms its function from partial sum accumulation to output merging. Different banks in the local output buffer have corresponding partial sum indexes. Results of two columns are accumulated to one bank if they have the same partial sum index. Meanwhile, the next activated OU sends outputs to the other accumulator (Accumulator2) for partial sum accumulation. Since OUs are activated sequentially, different OUs can always share the same two accumulators for partial sum accumulation and output merging. In future work, the ping-pong accumulators can be further strengthened with the self-adaptive OR-gate based approximate adder [13] to reduce power consumption, since additions happen frequently both in partial sum accumulation and output merging. Approximate computing incurs negligible accuracy loss in NN inference, thanks to the fault-tolerant property of neural networks [13], [14].

B. Spatial Input Dispatcher

Fig. 10 illustrates the spatial input dispatcher, which is composed of a selection unit and a distribution network.

1) Selection Unit: Since the weights in CIM cores are reordered, we design a selection unit to collect required inputs from the global input buffer to match the spatial workload mapping. The input matrix is divided in the OU-height granularity and stored in N_{Bank} banks of the global input buffer (bank width is H_{OU}). The selection unit first reads the corresponding input banks based on the weights stored in CIM. To match the weight order, the selection unit uses H_{Macro}/H_{OU} multiplexers to switch the input position. In the example of Fig. 10(b), Brick4 and Brick1 are selected and coalesced as the H_{Macro}-bit input of the current sub-matrix. In this way, input tiles are formed with the selected H_{Macro}-bit inputs in each cycle, and fed to the distribution network.

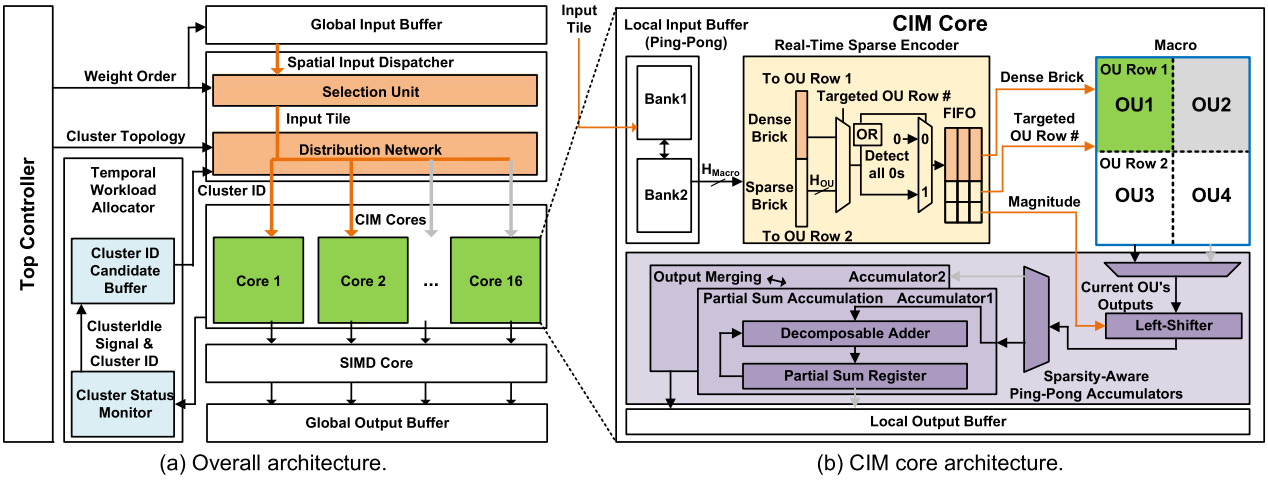


Fig. 9. SPCIM architecture design: (a) Overall architecture, featuring a spatial input dispatcher and a temporal workload allocator to solve the spatial and temporal under-utilization problems. (b) CIM core architecture that exploits OU-height input and weight sparsity (an example with four OUs per macro). For input sparsity, we employ a real-time sparse encoder to extract dense bricks from the input tile. For weight sparsity, we design sparsity-aware ping-pong accumulators to execute sequential partial sum accumulation and output merging.

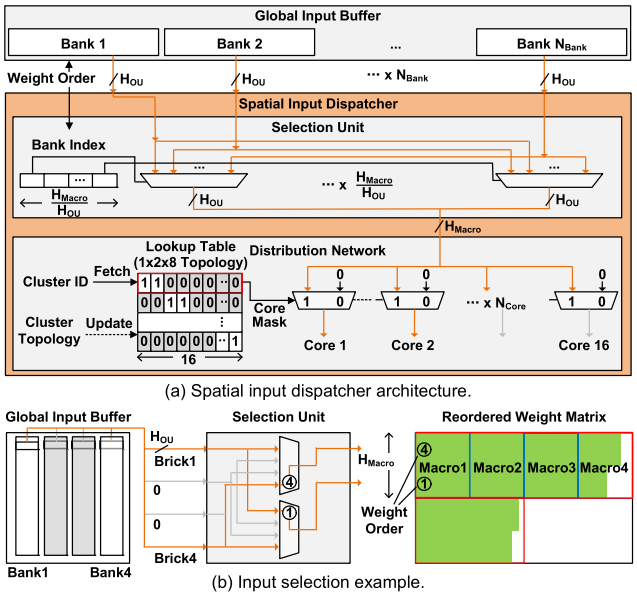


Fig. 10. Spatial input dispatcher: (a) Spatial input dispatcher architecture, including a selection unit to collect input tiles and a distribution network for CIM cluster reconfiguration. (b) Input selection example, where Brick4 and Brick1 are selected and coalesced to feed the reordered weight matrix in CIM.

2) Distribution Network: To construct the reconfigurable CIM cluster topology, we design a distribution network that assigns the right inputs to the cores. The key idea is always keeping the same input feeding for the cores in one cluster. The network has a lookup table generated from the cluster topology. Each entry of the table is a core mask that indicates the CIM cores belonging to the same cluster. Once the distribution network receives the index of the destination cluster (Cluster ID), it fetches one entry of the lookup table and multicasts the input tile to the corresponding CIM cores. CIM cluster reconfiguration can be realized during runtime, by updating the core masks in the lookup table.

The selection unit, distribution network, and CIM cores form a pipeline. The selection unit collects inputs from the global input buffer and sends input tiles to the distribution network. The distribution network multicasts the input tile

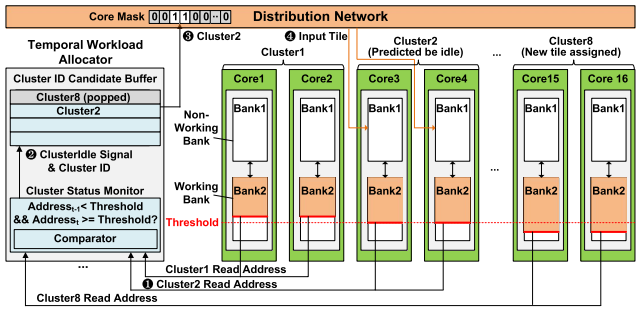


Fig. 11. Temporal workload allocator, which monitors each CIM cluster's computation status and dynamically assigns new input tiles to the non-working banks of idle clusters. New workload can be ready before the current computation finishes, so there is no halt time before the next computation.

to the corresponding CIM cores. The CIM cores store the received input tiles in the local input buffer and perform CIM computation. The latency of the first two stages is overlapped during the computation.

C. Temporal Workload Allocator

Fig. 11 demonstrates the temporal workload allocator for temporal workload mapping, which dynamically assigns input tiles to idle clusters. For clear demonstration, we divide the working process into four steps: ❶ Since the CIM core's local input buffer has a ping-pong bank structure, the allocator uses the working bank's read address, to monitor the real-time computation status. If the address exceeds a threshold, the allocator predicts the current input tile is nearly consumed and the corresponding cluster will become idle soon. ❷ A ClusterIdle signal is triggered and the CIM core's cluster ID is stored in the allocator's candidate buffer. The threshold is empirically set as 75% of the buffer depth to guarantee the new input tile feeding is completed before the current tile is totally consumed. ❸ When the global input buffer completes the current input tile feeding, the temporal workload allocator pops out a new cluster ID to request another input tile and guides the destination of the dispatcher's distribution network. ❹ In this way, the cores of the idle cluster prefetch a new

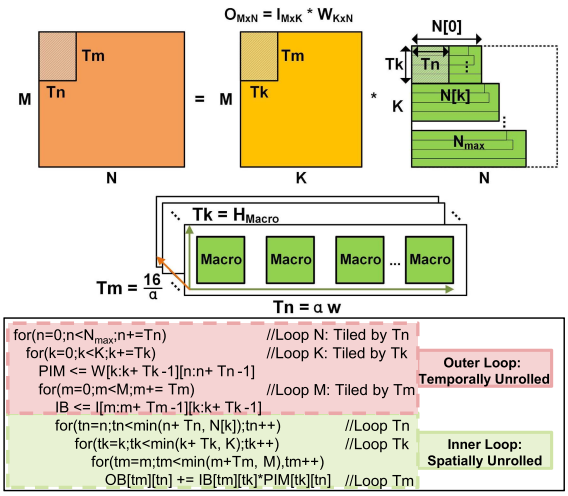


Fig. 12. NN layer mapping on the CIM cores. The inner loop utilizes the CIM cluster topology's three dimensions to spatially unroll computation in parallel. The outer loop reuses the weights in CIM and temporally unroll computation.

input tile and store it in the non-working bank of the local input buffer. New workload can be ready before the current computation finishes, so there is no halt time before the next computation. Each cluster can always keep busy, and different clusters' overall computation gets dynamically balanced, thus improving temporal utilization.

D. Layer Mapping and Work Flow

Fig. 12 shows how an entire layer is mapped onto SPCIM's CIM cores via a multi-level loop. SPCIM processes an NN layer in the matrix multiplication format: $O_{M\times N} = I_{M\times K} * W_{K\times N}$, where M, N, K are the matrix dimensions. Since the $K \times N$ weight matrix is compressed along the horizontal direction, CIM cores only need to store the compressed weights rather than the original $K \times N$ weight matrix. For every Tk $(=H_{Macro})$ rows, we only need to map $N[k]$ columns. Since we reorder the weight matrix based on the weight column count after compression, the last Tk rows have the largest weight parallelism in the N dimension (N_{max}), which determines the loop boundary in the N dimension. The matrix multiplication format makes SPCIM a universal architecture that is applicable to a variety of NNs including CNN. Although the proposed techniques are not specially designed for just CNN, the network can get large benefits from SPCIM's weight-stationary CIM architecture and sparsity balance optimizations.

Due to the limited on-chip resources, Loop N, K, M are tiled by the parameters Tn, Tk, Tm. The tiling produces inner and outer loops. The inner loops are spatially unrolled and mapped onto the CIM cores in parallel. The outer loops are temporally unrolled and scheduled by the top controller.

1) Inner Loop: By making use of the three parallel dimensions, the 16 CIM cores form a cube defined by Tn, Tk, Tm. Each macro stores $H_{Macro} \times w$ weights, performing H_{Macro} MACs (multiply-and-accumulation) for w elements in one row of the output matrix. The α macros in one cluster compute $\alpha \cdot w$ output elements simultaneously, so $Tk = H_{Macro}$, $Tn = \alpha \cdot w$. Tm equals the CIM cluster count $(\frac{16}{\alpha})$, so the CIM cores compute totally $\frac{16}{\alpha}$ rows of the output matrix.

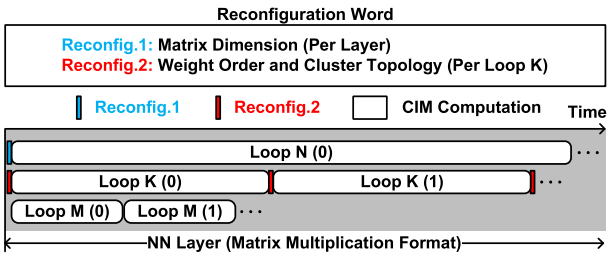


Fig. 13. SPCIM's work flow. To accommodate different layer parameters and sparsity patterns, we configure the top controller through reconfiguration words during runtime. Reconfig.1 stores the matrix dimensions and is configured before processing a new layer. Reconfig.2 stores the weight order and cluster information, and is configured between iterations of Loop K.

2) Outer Loop: We carefully design SPCIM's outer loop order as $N - K - M$. To exploit the weight-stationary [25] nature of CIM, loop M is set as the inner-most order. Loop K is placed just outside of Loop M to quickly complete the $M \times Tn$ output computation. Consequently, the partial sum storage in the output buffer can be released for a new round of $M \times Tn$ outputs. Since Loop N and K (Weight Dimensions) are placed outside of Loop M (Input Dimension), the weights are loaded into CIM only once and fully reused to compute all the M elements in one column of the output matrix.

Fig. 13 summarizes SPCIM's work flow, explaining how reconfiguration is performed during computation. To accommodate different layer parameters and sparsity patterns, we need to write new reconfiguration words into the top controller. Thus, we can manage on-chip components' behaviors during runtime. There are two kinds of reconfiguration words. Reconfig.1 stores the matrix dimensions, by transforming an NN layer to the matrix multiplication format through the *im2col* function. Reconfig.2 stores the weight order and cluster topology information. Since weight matrices are offline compressed and reordered, all weight order and cluster topology information for each Loop K are pre-determined. Weight matrix dimensions are also available before computation. Thus, the reconfiguration words are generated offline and stored in the off-chip memory.

During computation, Reconfig.1 and Reconfig.2 are fed to the top controller gradually. Reconfig.1 is configured when the CIM cores start processing a new layer, based on which the top controller schedules the outer loop. Reconfig.2 is configured between iterations of Loop K, at which time CIM cores are updated with a new set of $Tk \times Tn$ weights. Weight order guides the selection unit to collect inputs. According to the stored cluster topology, the top controller produces core masks for the current topology online, and updates the lookup table in the spatial input dispatcher.

Although CIM cores are updated during computation, CIM can still reduce data movement in this case. Due to the limited chip capacity and the growing size of modern NN models, it is impractical to always store all weights of an NN model on chip. If the model size exceeds the chip capacity, reloading weights between iterations is inevitable. As illustrated in Fig. 13, weight reloading happens between iterations of Loop K. Nevertheless, during an iteration of

Loop K, the weights are stored stationarily in CIM, without feeding repetitively from weight buffers to MAC (Multiply-Accumulation) units like digital architectures. The on-chip weight data movement is largely reduced thanks to the CIM architecture. Besides, we employ weight-stationary dataflow to ensure weights are loaded into PIM only once, so weight loading only consumes 0.60%∼3.28% energy and 0.85%∼4.82% latency in the end-to-end evaluations on benchmarks.

V. Evaluation

A. Experiment Setup

1) Accelerator Implementation: We implement an SPCIM accelerator in the 28nm technology, with a 256KB global input buffer, 128KB global output buffer, and 16 CIM cores. Each CIM core has a 4KB local input buffer and a 2KB local output buffer. The CIM macro specifications are based on a state-of-the-art sparse CIM accelerator ISSCC'21-15.2 [27]. The macro size is 128×128 and the OU size is 32×16. The OU limitation is taken into account, so it satisfies our practical CIM accelerator setting. The other RTL logic is synthesized by Synopsys Design Compiler for area estimation. Performance measurement is conducted on Synopsys VCS and considers all on-chip execution time including weight update. Power consumption is evaluated by PrimeTime PX.

2) Benchmarks: We select AlexNet [11], VGG-19 [18], ResNet-50, ResNet-152 [6], and DenseNet-169 [8] as benchmarks. The inputs and weights are quantized to 8b for all layers. The benchmarks are pruned to follow the OU-height sparsity pattern in Section III-B, and retrained to maintain accuracy. All models are pruned to reach 80% weight sparsity, so the difference in performance and energy consumption stems mainly from the benchmarks' structural differences. The OU-height level input sparsity varies across different input images and benchmarks. For each benchmark, the averaged input sparsity ratio is 30∼50%. The benchmarks' layers are transformed to the matrix multiplication format for execution on CIM.

3) Methodology: First, Section V-B summarizes SPCIM's implementation specifications. Then, in Section V-C, we go deeply into SPCIM's behaviors on benchmarks to evaluate how performance and energy efficiency are improved with SPCIM's different techniques. To further analyze how the benefits are scaled with different weight sparsity ratios and number of CIM macros, we conduct sensitivity studies in Section V-D. Finally, Section V-E provides a detailed comparison with state-of-the-art CIM accelerators.

B. SPCIM Implementation Summary

Fig. 14 shows SPCIM's breakdown analysis. Operating at 1V, 232MHz, SPCIM's total area and power are 2.75mm^2 and 95.00mW. The CIM cores, as the key computing units, consume 56.63% area and 58.59% power. The global input buffer and global output buffer, due to their large capacity, take up 41.04% area and 37.20% power. The spatial input dispatcher and temporal workload allocator are implemented with lightweight control and configuration logic, so they only cost 0.63+0.25 = 0.88% in area and 0.86+0.50 = 1.36% in power.

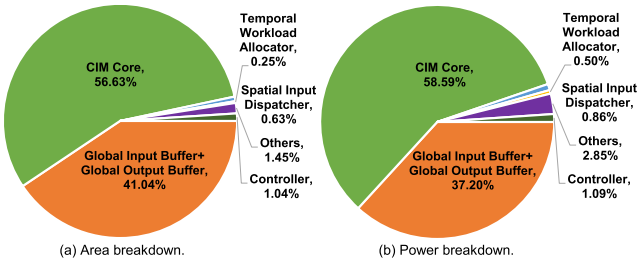


Fig. 14. Area and power breakdown of SPCIM. The proposed techniques incur negligible overhead (0.63+0.25=0.88% in area and 0.86+0.50=1.36% in power) due to their lightweight control and configuration logic.

C. Technique Breakdown Analysis

1) Comparison Baselines: To evaluate the benefits and overhead of the proposed techniques, we first implement a baseline sparse CIM accelerator by removing the spatial input dispatcher and temporal workload allocator from SPCIM. The baseline has 16 CIM cores organized in a fixed 2D-mesh topology as described in Fig. 5(a), which is similar to Ref. [27]. We also implement two variants of our accelerator to evaluate the benefits and overhead of each proposed technique. The variant Base+CR (Cluster Reconfiguration) equips the baseline with the distribution network to enable the reconfigurable cluster topology of $1\times1\times16$, $1\times2\times8$, $1\times4\times4$, $1\times8\times2$, and $1\times16\times1$, with the clustering factor ranging from 1 to 16. The variant Base+CR+WR (Weight Reordering) further enhances the baseline by adding the selection unit and reordering the weight matrix for better spatial workload mapping. SPCIM finally adds the temporal workload allocator to support temporal workload mapping.

2) Utilization Metrics: To demonstrate our improvement in spatial-temporal multi-macro utilization, we plot the detailed utilization rate in Fig. 15. The spatial utilization is defined as the actual performance divided by the ideal performance if all macros are fully mapped with weights. The temporal utilization is defined as the actual performance divided by the ideal performance if every macro is performing computation all the time without macro stall. SPCIM's distribution network and selection unit support the spatial workload mapping to improve spatial utilization, so we compare the spatial utilization of the baseline, Base+CR, and Base+CR+WR. SPCIM's temporal workload allocator realizes the temporal workload mapping for higher temporal utilization, so we only compare the temporal utilization of Base+CR+WR and SPCIM.

3) Evaluation for Cluster Reconfiguration: Fig. 16 shows the speedup and normalized energy of different architecture variants on the benchmarks. With cluster reconfiguration, the speedup ranges from 1.16× to 2.26× (1.57× on average), and the average spatial utilization grows from 36.3% to 57.1%. Since the convolutional layers of DenseNet-169 usually have fewer kernels than those of other networks, the N dimension of its weight matrix is rather small, especially after horizontal compression of dense weights. Therefore, the baseline's CIM cores have a higher chance to suffer from spatial under-utilization on DenseNet-169, and the spatial utilization

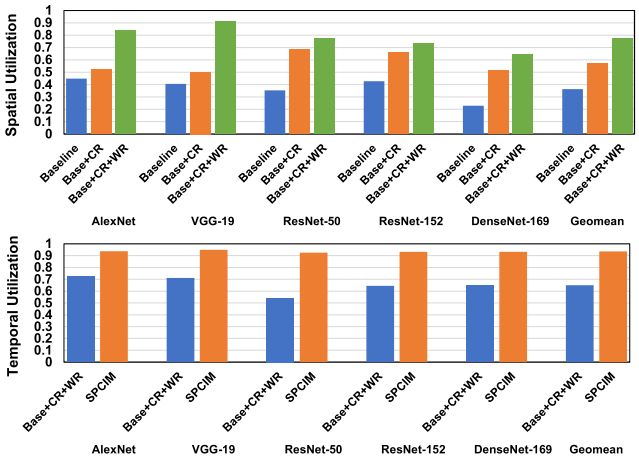


Fig. 15. Spatial and temporal utilization of the different architecture variants. SPCIM's cluster reconfiguration and weight reordering improve spatial utilization, while the dynamic input allocation improves temporal utilization.

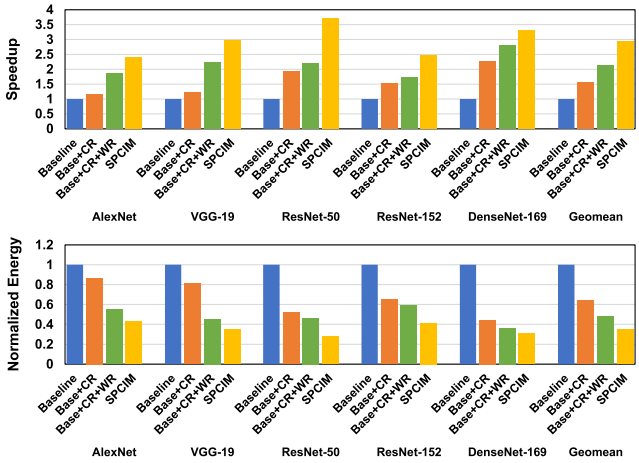


Fig. 16. Speedup and normalized energy of different architecture variants that represent the benefits of SPCIM's cluster reconfiguration, weight reordering, and dynamic input allocation techniques.

is only 22.9%. After cluster reconfiguration is applied, the under-utilization problem is greatly alleviated and the spatial utilization is raised to 51.8%. As a result, DenseNet-169 obtains the largest speedup 2.26× from cluster reconfiguration.

4) Evaluation for Weight Reordering: With the joint use of cluster reconfiguration and weight reordering, the overall speedup is improved to 1.72×∼2.80× (2.13× on average). VGG-19 obtains the highest extra benefit from weight reordering (1.81×). Since VGG-19 has more channels than those of others, the K dimension of its weight matrix is larger. Therefore, weight reordering can sort more weight groups, and the weight parallelism in a cluster becomes much similar. The spatial utilization is largely boosted from 50.3% to 91.1%. On the contrary, ResNet-50 and ResNet-152 have many bottleneck layers that have 1×1 kernel size and fewer input channels. These layers exist few weight groups and the distribution of dense weights is still imbalanced after weight reordering. Thus, weight reordering's speedup is lower on ResNet-50 and ResNet-152 (1.13× and 1.12× respectively).

5) Evaluation for Dynamic Input Allocation: Dynamic input allocation delivers extra performance gains, ranging from 1.18× to 1.70× on the benchmarks. When dynamic input allocation is applied, the overall speedup is further raised to 2.42×∼3.72× and 2.94× on average. ResNet-50 and ResNet-152 obtain higher improvement from dynamic input allocation than the other networks, by 1.70× and 1.44×, respectively, because they have many normalization layers that make it easier to find sparse inputs. Therefore, the execution time gap of the densest and sparsest inputs is enlarged. In the baseline, macros have to wait more time for the macro with the densest inputs to complete computation, and our temporal balance approach achieves more speedup. After dynamic input allocation is applied, ResNet-50 and ResNet-152's temporal utilization increases from 54.3%, 64.6% to about 93%, demonstrating that the computation time of different clusters is quite close and macro stall is almost eliminated with well-balanced inputs to different clusters.

6) Layerwise Analysis on DenseNet-169: To evaluate how benefits vary under different layer dimensions, we conduct a layerwise analysis on DenseNet-169. Fig. 17 presents evaluation results on some typical layers of DenseNet-169. For most layers of DenseNet-169 such as Conv7 × 7 and dense blocks' Conv1 × 1 layers, they have fewer kernels than those of other networks (e.g., 64, 32). After horizontal compression of dense weights, the kernel number as well as the weight matrix size becomes even smaller. Thus, the spatial under-utilization problem is more severe on these layers in the baseline, and SPCIM's cluster reconfiguration can gain more benefits. For other layers such as transition layers and dense blocks' Conv3 × 3 layers, they have more kernel counts. The under-utilization problem is less severe, so SPCIM has lower speedup and energy saving.

The proposed techniques improve the spatial-temporal multi-macro utilization significantly, with negligible latency and energy overhead. The temporal workload allocator monitors the computation status, and controls the spatial input dispatcher to prefetch new workload into the non-working bank before the current computation finishes. The input dispatcher receives the destination cluster index from the workload allocator, and multicasts the input tile to the corresponding CIM cores. In this way, the workload prefetch and CIM computation form a pipeline, where the latency of workload prefetch is mostly overlapped during the computation and only takes a small proportion of the overall latency (3.40%). Besides, since the proposed techniques' power is low (1.36% overhead for the spatial input dispatcher and temporal workload allocator), the normalized energy also decreases with the employment of spatial and temporal workload mapping. The mean energy saving over the baseline is 2.86×, with the highest of 3.63× on ResNet-50 and the lowest of 2.35× on AlexNet. SPCIM's cluster reconfiguration, weight reordering, and dynamic input allocation contribute to average reduction of 1.56×, 1.34×, and 1.37×, respectively.

D. Sensitivity and Generalization Studies

We evaluate SPCIM with different weight sparsity ratios and number of CIM macros, to analyze how the algorithm and architectural parameters impact our techniques' benefits. We also study the efficiency of SPCIM's different techniques on lightweight NNs that have limited weight sparsity ratio.

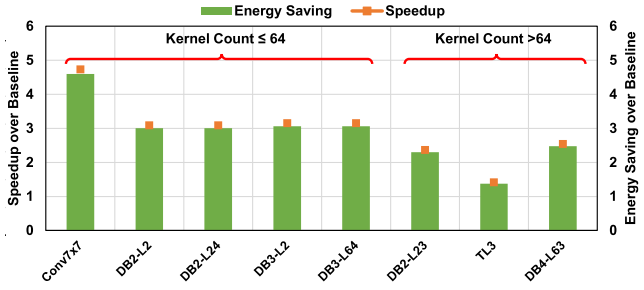


Fig. 17. Speedup and energy savings over the baseline sparse CIM accelerator on typical layers of DenseNet-169.

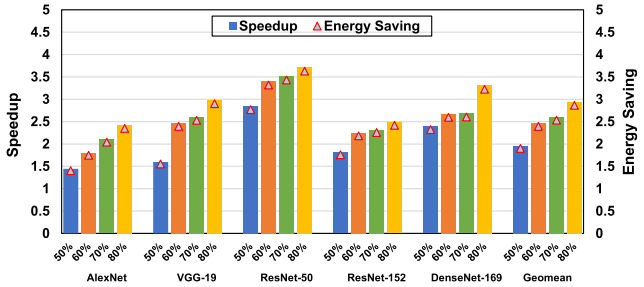


Fig. 18. Speedup and energy saving over the baseline on benchmarks with different weight sparsity ratios (from 50% to 80%). The smaller weight matrix size and enlarged imbalance at a higher sparsity ratio make the baseline's spatial under-utilization more severe. SPCIM's cluster reconfiguration and weight reordering can gain more benefits under higher weight sparsity.

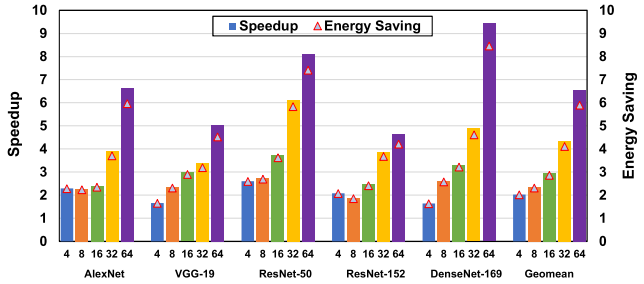


Fig. 19. Speedup and energy saving over the baseline with different number of CIM macros (from 4 to 64). When more macros are organized in a fixed 2D-mesh topology, the baseline is easier to suffer from spatial under-utilization. SPCIM's cluster topology can be reshaped to make adaptive exploitation of input parallelism and weight parallelism.

1) Comparison Baselines: In the sensitivity study of weight sparsity ratio, the baseline architecture is the same as that in Section V-C. 16 CIM cores are organized in a fixed 2D-mesh topology, without support for spatial and temporal workload mapping. We prune each benchmark with different thresholds to reach more weight sparsity ratios, and evaluate SPCIM's benefits over the baseline on these benchmarks respectively. In the sensitivity study of the macro count, we implement a series of baselines under different macro counts, in accordance with SPCIM's settings. The baselines have the same macro counts and organize the CIM macros in a fixed topology that can achieve the best utilization rate averagely on five benchmarks. For the macro count from 4 to 64, the macro topology is 2×2, 2×4, 4×4, 8×4, and 8×8 respectively. Each CIM macro is still placed in one CIM core, and the related control logic scales with the macro count. The baseline in the generalization study of lightweight NNs is also the same as that in Section V-C.

2) Weight Sparsity Ratio: In SPCIM, the weight sparsity ratio impacts the gain from cluster reconfiguration and weight reordering. We prune the benchmarks to 50%, 60%, 70%, and 80% weight sparsity with the OU-height sparsity pattern. Fig. 18 shows SPCIM's speedup and energy savings over the baseline on benchmarks with different sparsity ratios. SPCIM achieves $1.44\times\sim2.84\times$ speedup and $1.40\times\sim2.77\times$ energy saving at 50% weight sparsity. The speedup and energy saving are raised to $2.42\times\sim3.72\times$ and $2.35\times\sim3.63\times$ at 80% sparsity. The higher sparsity ratio leads to a smaller weight matrix size after compression, making the baseline's under-utilization problem more severe. Thus, SPCIM can gain more benefits from cluster reconfiguration. Moreover, the growth of sparsity ratio enlarges the imbalance between the densest and sparsest weight groups. At a higher sparsity ratio, the matrix shape becomes more irregular and only a smaller proportion of CIM macros can be mapped with weights in the baseline. Weight reordering sorts weight groups to achieve similar weight parallelism in a cluster, so the sub-matrices' shapes are regular and more weights can be mapped onto CIM macros. Besides more benefits from cluster reconfiguration, the weight reordering can also bring more speedup and energy saving under higher weight sparsity.

3) Number of CIM Macros: We evaluate SPCIM with different number of CIM macros, to analyze the scaling of performance and energy efficiency. Fig. 19 demonstrates that as the macro number increases, the speedup over the baseline also increases (from $2.03\times$ to $6.54\times$ on average). When more macros are organized in a fixed 2D-mesh topology, it's easier for the baseline accelerator to suffer from spatial under-utilization. For SPCIM, the cluster topology can be reshaped to make adaptive exploitation of input parallelism and weight parallelism. As a result, the growing macro number can convert to the extended input parallelism and the spatial utilization is well maintained. DenseNet-169 achieves the greatest speedup with 64 CIM macros ($9.43\times$), since its weight matrix sizes are smaller than those of other networks. Therefore, the baseline has a more severe under-utilization problem on DenseNet-169 when the macro number increases. Since SPCIM can always reconfigure itself to match diverse workload parallelism, more benefits are gained in this low-performance baseline. Thanks to the low power overhead of SPCIM's sparsity balance support, the energy saving also scales well with the macro count, increasing from $2.01\times$ to $5.90\times$ on average.

4) Generalization on Lightweight NNs: The CIM based NN accelerators are mostly used at the edge devices, where the lightweight models are frequently used. Different from normal models, lightweight models are difficult to prune with high sparsity. We select MobileNet-v2 and EfficientNet-B0 as the representation of lightweight models, to study the efficiency of SPCIM's different proposed techniques on such dense networks. MobileNet-v2 and EfficientNet-B0 are pruned to reach 20% weight sparsity to maintain accuracy. Fig. 20 shows that SPCIM is still efficient for lightweight models. First, the original matrix sizes of lightweight models are much smaller than those of conventional networks. Therefore, the spatial under-utilization problem remains in the baseline even without a high weight sparsity ratio. Second, the ReLU function still

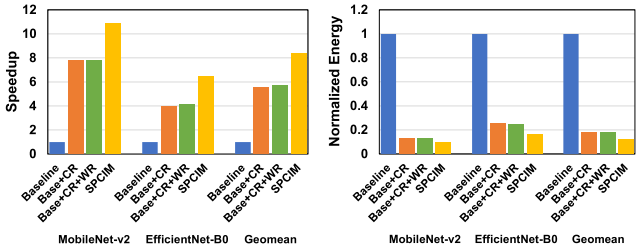


Fig. 20. Speedup and normalized energy of different architecture variants on lightweight models (MobileNet-v2 and EfficientNet-B0).

exists in lightweight models, so input sparsity can also be exploited. The macros' different computation time causes temporal under-utilization. The cluster reconfiguration and dynamic input allocation improve the multi-macro utilization, while the weight reordering's benefit is reduced due to the less imbalanced dense weight distribution under low sparsity. Overall, SPCIM achieves 8.39× speedup and 8.14× energy saving on average.

E. Comparison With State-of-the-Art Works

TABLE II shows a detailed comparison with state-of-the-art CIM accelerators. ISSCC'20-14.3 [28] skips all-zero inputs to accelerate sparse activation computation, but weight sparsity only contributes to power saving by powering off ADCs, with no reduction to execution time. ISSCC'21-15.2 [27] further compresses block-wise sparse weight matrices to reduce storage and execution time, but it only focuses on the sparsity processing on a single CIM macro. Macros are organized in a fixed topology, with no consideration of sparsity balance and multi-macro workload mapping. ISSCC'22-11.8 [21] is a state-of-the-art SRAM-CIM accelerator with time-domain accumulation but without sparsity support. Different from previous work, SPCIM is a practical sparse CIM accelerator that supports reconfigurable multi-macro organization and spatial-temporal workload balance. SPCIM's dataflow and architecture are generic and scalable, which can be applied to other CIM accelerators. Thus, we enhance two state-of-the-art CIM macros (ISSCC'21-15.2 [27] and ISSCC'22-11.8 [21]) with our sparsity-balanced dataflow and architecture, and evaluate their performance and energy efficiency (see the last two columns of TABLE II).

To compare with ISSCC'21-15.2 [27], we implement SPCIM in the 65nm technology and set the same working point (0.65V for digital and 1.0V for CIM macro, 37.5MHz). The other implementation details are in accordance with the settings in Section V-A. Ref. [27] achieves 2.75TOPS/W energy efficiency at INT8/4 (8b input and 4b weight). Thanks to the improved CIM utilization from multi-macro workload balance and reconfigurable organization, SPCIM delivers 5.29TOPS/W energy efficiency at INT8/4, which is 1.92× higher than the reported result in Ref. [27].

To compare with ISSCC'22-11.8 [21], we implement SPCIM in the 28nm technology and set the working point as Ref. [21]'s highest efficiency point (128MHz, 0.65V). In this evaluation, we replace SPCIM's CIM macros with Ref. [21]'s, whose array size is 128 × 1024 and OU size is 64 × 64. SPCIM organizes 16 SRAM-CIM arrays with

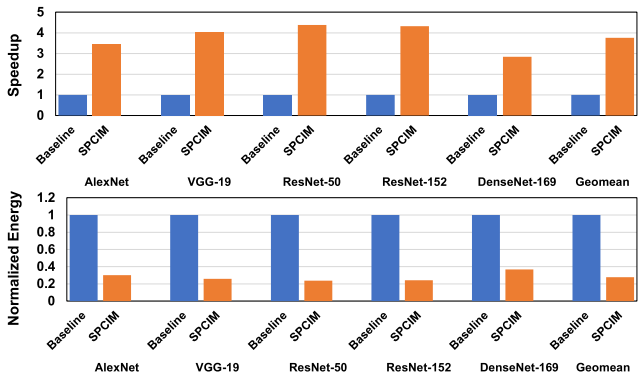


Fig. 21. SPCIM's speedup and normalized energy over ISSCC'21-15.1 [10].

our reconfigurable cluster topology, and inputs to different clusters are dynamically balanced. The OU-level sparsity support brings speedup and energy saving from zero skipping. The multi-macro organization and workload balance ensure CIM macros' high utilization under sparsity exploitation. After our techniques are applied, the energy efficiency is raised to 127.23TOPS/W, which is 5.59× higher than the reported result in Ref. [21].

Compatibility With Related Techniques: We provide further discussions on prior techniques related to SPCIM's reconfiguration and sparsity optimization. ISSCC'21-15.1 [10] provides flexibility for dense NN models, but its extension dimension incurs severe under-utilization problem when sparsity is exploited. RePIM [20] focus on just single-macro sparsity, which is orthogonal to our sparsity balance techniques for multiple CIM macros. Our SPCIM's techniques can be applied to the two prior works for better sparsity exploitation.

ISSCC'21-15.1 [10] extends its macros in the output-tensor depth or input-tensor depth. However, to support the input-tensor depth extension, outputs of adjacent CIM macros are accumulated via face-to-face connections. When input sparsity is exploited, temporal under-utilization arises every time the inter-macro accumulation happens. The CIM macros' computation times are different due to the input sparsity, so the slower macro has to wait frequently during each accumulation. To compare with Ref. [10], we implement SPCIM in the 22nm technology, set the same working point (0.8V, 200MHz), and replace SPCIM's CIM macros with Ref. [10]'s. The baseline is the original architecture that extends CIM macros in the output-tensor depth or input-tensor depth, without sparsity exploitation. We strengthen the architecture with SPCIM's reconfigurable clustering that improves spatial-temporal multi-macro utilization for better sparsity exploitation. SPCIM delivers 3.76× average speedup and 3.61× energy saving on five benchmarks, as plotted in Fig. 21.

The main contribution of our paper is the generic and scalable sparsity-balanced dataflow and architecture, which can be applied to other CIM accelerators like RePIM [20] and improve their energy efficiency. RePIM only discusses how to exploit NN sparsity on a single CIM macro. Our focus is how to efficiently organize multiple CIM macros to alleviate sparsity imbalance across macros and improve multi-macro utilization. To compare with RePIM, we implement SPCIM in the 32nm technology, and replace SPCIM's

TABLE II
COMPARISON WITH STATE-OF-THE-ART CIM ACCELERATORS

	ISSCC'20-14.3 [28]	ISSCC'21-15.2 [27]	ISSCC'22-11.8 [21]	This Work	
				Based on Ref. [27]	Based on Ref. [21]
OU-Level Sparsity Support	Input Skip, Weight Gating	Input Skip, Weight Skip	×	**Input Skip, Weight Skip**	
Reconfig. Multi-Macro Organization	×	×	×	✓	
Spatial-Temporal Workload Balance	×	×	×	✓	
Input Precision	INT2/4/6/8	INT2/4/6/8	INT4/8	INT2/4/6/8	INT4/8
Weight Precision	INT4/8	INT1∼INT8	INT4/8	INT1∼INT8	INT4/8
Technology	65nm	65nm	28nm	65nm	28nm
Supply Voltage (V)	0.9∼1.05	0.62∼1.0	0.65∼0.9	0.62∼1.0	0.65∼0.9
Frequency (MHz)	50∼100	25∼100	128∼152	25∼100	128∼152
Area (mm^2)	5.66	8.32	-	14.82	-
Power (mW)	31.8∼65.2	18.6∼84.1	46.02∼66.81*3	43.05	88.80
Performance (GOPS)*1	18.12 (INT4/8)	28.00 (INT8/4) 112.00 (INT8/4, Norm.)*4	1241.2 (INT8) 2482.4 (INT8, Norm.)*4	227.7 (INT8/4)	11298.1 (INT8)
Energy Efficiency (TOPS/W)*1,2	3.56 (INT4/8)	2.75 (INT8/4)	22.78 (INT8)*3	5.29 (INT8/4)	127.23 (INT8)

*1: One operation (OP) represents one multiplication or one addition. ISSCC'22-11.8 only has dense evaluation results, while others report sparse evaluation results.
*2: Accelerator energy efficiency includes all on-chip components.
*3: On-chip buffers are included for fair comparison. The global+local buffer capacity is 480KB in total, the same as the SPCIM accelerator.
*4: Normalized to the same dense peak throughput as SPCIM.

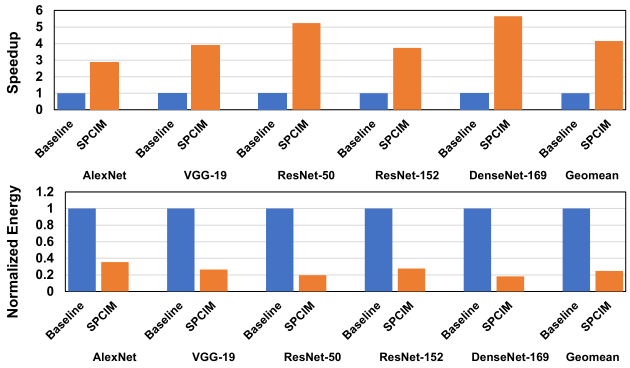


Fig. 22. SPCIM's speedup and normalized energy over RePIM [20].

CIM macros and sparsity exploitation method with RePIM's. The baseline organizes macros in a fixed topology, with no consideration of sparsity balance and multi-macro workload mapping. Enhanced with our sparsity balance support, SPCIM achieves 4.16× speedup and 4.04× energy saving on average, as shown in Fig. 22.

VI. CONCLUSION

In this work, we propose the SPCIM accelerator that solves the spatial and temporal under-utilization problems of practical CIM-base sparse NN acceleration. The reconfigurable CIM cluster topology and weight reordering tackle the spatial under-utilization. The OU-height weight sparsity pattern and dynamic input allocation deal with the temporal under-utilization. The SPCIM architecture features a spatial input dispatcher and a temporal workload allocator, achieving considerable speedup and energy saving over the baseline. The proposed dataflow and architecture techniques are generic and scalable, which can be easily applied to other sparse CIM accelerators for higher performance and energy efficiency.

REFERENCES

[1] W.-H. Chen et al., "A 65 nm 1Mb nonvolatile computing-in-memory ReRAM macro with sub-16ns multiply-and-accumulate for binary DNN AI edge processors," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Jun. 2018, pp. 494–496.

[2] P. Chi et al., "Prime: A novel processing-in-memory architecture for neural network computation in ReRAM-based main memory," in *Proc. 43rd Int. Symp. Comput. Archit.*, 2016, pp. 27–39.

[3] Q. Dong et al., "15.3 A 351 TOPS/W and 372.4 GOPS compute-in-memory SRAM macro in 7 nm FinFET CMOS for machine-learning applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2020, pp. 242–244.

[4] L. Fick, S. Skrzyniarz, M. Parikh, M. B. Henry, and D. Fick, "Analog matrix processor for edge AI real-time video analytics," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2022, pp. 260–262.

[5] R. Guo et al., "A 5.99-to-691.1 TOPS/W tensor-train in-memory-computing processor using bit-level-sparsity-based optimization and variable-precision quantization," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 242–244.

[6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[7] K. Hsu et al., "A study of array resistance distribution and a novel operation algorithm for WOx ReRAM memory," in *Proc. Int. Conf. Solid State Devices Mater. (SSDM)*, 2015, pp. 1168–1169.

[8] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 4700–4708.

[9] J.-M. Hung et al., "An 8-Mb DC-current-free binary-to-8b precision ReRAM nonvolatile computing-in-memory macro using time-space-readout with 1286.4–21.6TOPS/W for edge-AI devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2022, pp. 1–3.

[10] H. Jia et al., "A programmable neural-network inference accelerator based on scalable in-memory computing," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Sep. 2021, pp. 236–238.

[11] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2012, pp. 84–90.

[12] M.-Y. Lin et al., "DL-RSIM: A simulation framework to enable reliable ReRAM-based accelerators for deep learning," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2018, pp. 1–8.

[13] B. Liu et al., "A 22 nm, 10.8 μW/15.1 μW dual computing modes high power-performance-area efficiency domained background noise aware keyword-spotting processor," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 67, no. 12, pp. 4733–4746, Dec. 2020, doi: 10.1109/TCSI.2020.2997913.

[14] B. Liu et al., "More is less: Domain-specific speech recognition microprocessor using one-dimensional convolutional recurrent neural network," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 4, pp. 1571–1582, Apr. 2022.

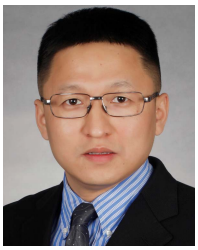
[15] A. Shafiee et al., "ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars," in *Proc. ACM/IEEE 43rd Annu. Int. Symp. Comput. Archit. (ISCA)*. Piscataway, NJ, USA: IEEE Press, Jun. 2016, pp. 14–26.

[16] X. Si et al., "24.5 A twin-8T SRAM computation-in-memory macro for multiple-bit CNN-based machine learning," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2019, pp. 396–398.
[17] X. Si et al., "15.5 A 28 nm 64Kb 6T SRAM computing-in-memory macro with 8b MAC operation for AI edge chips," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2020, pp. 246–248.
[18] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.
[19] J.-W. Su et al., "A 28 nm 384 kb 6T-SRAM computation-in-memory macro with 8b precision for AI edge chips," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 250–252.
[20] C.-Y. Tsai, C.-F. Nien, T.-C. Yu, H.-Y. Yeh, and H.-Y. Cheng, "RePIM: Joint exploitation of activation and weight repetitions for in-ReRAM DNN acceleration," in *Proc. 58th ACM/IEEE Design Autom. Conf. (DAC)*, Dec. 2021, pp. 589–594.
[21] P.-C. Wu et al., "A 28 nm 1Mb time-domain computing-in-memory 6T-SRAM macro with a 6.6ns latency, 1241 GOPS and 37.01 TOPS/W for 8b-MAC operations for edge-AI devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2022, pp. 1–3.
[22] C.-X. Xue et al., "A 1Mb multibit ReRAM computing-in-memory macro with 14.6ns parallel MAC computing time for CNN based AI edge processors," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2019, pp. 388–390.
[23] C.-X. Xue et al., "A 22 nm 2Mb ReRAM compute-in-memory macro with 121–28 TOPS/W for multibit mac computing for tiny AI edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2020, pp. 244–246.
[24] C.-X. Xue et al., "16.1 A 22 nm 4Mb 8b-precision ReRAM computing-in-memory macro with 11.91 to 195.7 TOPS/W for tiny AI edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2021, pp. 245–247.
[25] T.-J. Yang and V. Sze, "Design considerations for efficient deep neural networks on processing-in-memory accelerators," in *IEDM Tech. Dig.*, Dec. 2019, pp. 1–22.
[26] T.-H. Yang et al., "Sparse ReRAM engine: Joint exploration of activation and weight sparsity in compressed neural networks," in *Proc. ACM/IEEE 46th Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2019, pp. 236–249.
[27] J. Yue et al., "A 2.75-to-75.9 TOPS/W computing-in-memory NN processor supporting set-associate block-wise zero skipping and ping-pong CIM with simultaneous computation and weight updating," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 238–240.
[28] J. Yue et al., "A 65 nm computing-in-memory-based CNN processor with 2.9-to-35.8 TOPS/W system energy efficiency using dynamic-sparsity performance-scaling architecture and energy-efficient inter/intra-macro data reuse," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2020, pp. 234–236.

Yiqi Wang received the B.S. degree from the School of Integrated Circuits, Tsinghua University, Beijing, China, in 2021, where he is currently pursuing the Ph.D. degree. His research interests include deep learning, in-memory computing, and computer architecture.

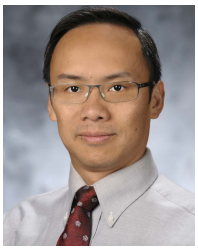
Fengbin Tu (Member, IEEE) received the B.S. degree from the School of Electronic Engineering, Beijing University of Posts and Telecommunications, Beijing, China, in 2013, and the Ph.D. degree from the Institute of Microelectronics, Tsinghua University, Beijing, in 2019.

He is currently a Post-Doctoral Researcher at the Scalable Energy-Efficient Architecture Laboratory (SEAL), Department of Electrical and Computer Engineering, University of California at Santa Barbara, Santa Barbara, CA, USA. He has published two books, such as *Architecture Design and Memory Optimization for Neural Network Accelerators* and *Artificial Intelligence Chip Design*. His research works appeared at top conferences and journals on integrated circuits and computer architecture, including ISSCC, IEEE JOURNAL OF SOLID-STATE CIRCUITS, DAC, ISCA, MICRO, and ASPLOS. His research interests include computer architecture, reconfigurable computing, in-memory computing, and deep learning. His AI Chip Thinker won the 2017 ISLPED Design Contest Award. His Ph.D. thesis won the Tsinghua Excellent Dissertation Award.



Leibo Liu (Senior Member, IEEE) received the B.S. degree in electronic engineering from Tsinghua University, Beijing, China, in 1999, and the Ph.D. degree from the Institute of Microelectronics, Tsinghua University, in 2004. He currently works as a Professor with the Institute of Microelectronics, Tsinghua University. His research interests include reconfigurable computing, mobile computing, and VLSI DSP.

Shaojun Wei (Fellow, IEEE) was born in Beijing, China, in 1958. He received the Ph.D. degree from the Faculte Polytechnique de Mons, Belgium, in 1991. He became a Professor with the Institute of Microelectronics, Tsinghua University, in 1995. His main research interests include VLSI SoC design, EDA methodology, and communication ASIC design. He is a Senior Member of the Chinese Institute of Electronics (CIE).



Yuan Xie (Fellow, IEEE) received the B.S. degree in electronic engineering from Tsinghua University, Beijing, China, in 1997, and the M.S. and Ph.D. degrees in electrical engineering from Princeton University, Princeton, NJ, USA, in 1999 and 2002, respectively.

He was an Advisory Engineer with IBM Microelectronics Division, Burlington, NJ, from 2002 to 2003. He was a Full Professor with Pennsylvania State University, State College, PA, USA, from 2003 to 2014. He was a Visiting Researcher with the Interuniversity Microelectronics Centre (imec), Leuven, Belgium, from 2005 to 2007 and in 2010. He was a Senior Manager and a Principal Researcher with the AMD Research China Laboratory, Beijing, from 2012 to 2013. He is currently a Professor with the Department of Electrical and Computer Engineering, University of California at Santa Barbara, Santa Barbara, CA, USA. His research interests include VLSI design, electronic design automation (EDA), computer architecture, and embedded systems.

Dr. Xie is an AAAS Fellow and an ACM Fellow. He is an Expert in computer architecture who has been inducted to the IEEE/ACM International Symposium on Computer Architecture (ISCA)/the IEEE/ACM International Symposium on Microarchitecture (MICRO)/IEEE International Symposium on High-Performance Computer Architecture (HPCA) Hall of Fame.

Shouyi Yin (Member, IEEE) received the B.S., M.S., and Ph.D. degrees in electronic engineering from Tsinghua University, Beijing, China, in 2000, 2002, and 2005, respectively.

He worked as a Research Associate with Imperial College, London, U.K. He is currently a Full Professor and the Vice Director of the Institute of Microelectronics, Tsinghua University. He has published more than 100 journal articles and more than 50 conference papers. His research interests include reconfigurable computing, AI processors, and high level synthesis. He has served as a Technical Program Committee Member in the top VLSI and EDA conferences, such as A-SSCC, MICRO, DAC, ICCAD, and ASPDAC. He is an Associate Editor of the IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—I: REGULAR PAPERS, *ACM TRETS*, and *Integration, the VLSI Journal*.